

System Design Document

Team: LionsList

Members: Isha Chaudhry, Raymond Au, Gurleen Sandhu, Andrei Lopez, Justin Valle Amaya

York University

EECS3311: Software Design

Iir Dema

March 23, 2026

Table of Contents

1. Introduction.....	3
2. System Environment.....	4
3. System Architecture.....	5
4. System Decomposition.....	6
4.1 Frontend Components.....	6
4.2 Backend Components.....	6
4.3 Database Models.....	7
5. CRC Cards.....	8
6. Error Handling Strategy.....	10
6.1 Invalid User Input.....	10
6.2 Authentication and Authorization Errors.....	10
6.3 Database Errors.....	11
6.4 Network and API Communication Errors.....	11
6.5 Data Integrity Protection.....	12
6.6 Logging and Debugging.....	12

1. Introduction

LionsList is a student-focused web-based marketplace designed exclusively for York University to buy and sell academic materials within the York University Community.

General marketplaces such as Kijiji and Facebook Marketplace do allow people to list textbooks and academic supplies but are not specifically aimed at university students. These platforms do not have the ability to filter by course, nor the verification of the users. Consequently, students frequently come across irrelevant listings, unreliable buyers, scams, and low value offers on these platforms.

The official York University bookstore is where students can get the academic materials they need. However, prices are usually fixed at high retail prices paired with a strict refund, return, and exchange policy. Online forums like the unofficial York University subreddit provide a medium for students to communicate, but lack structured listings, filter functionality, and user verification.

LionsList addresses these limitations by providing:

- York University Exclusive Access
 - Course-code Filtering
 - User Verification
 - Searchable Academic Materials Listings
 - Flexible Student Pricing
-

2. System Environment

Runtime Environment

- Node.js (LTS version)
- MongoDB database
- Web browser for accessing the client application

Programming Technologies

- Frontend: React, Axios HTTP client, JavaScript / HTML / CSS
- Backend: Node.js, Express.js framework, Mongoose ODM for MongoDB
- Database: MongoDB

Project Structure

The repository contains two main components:

client/

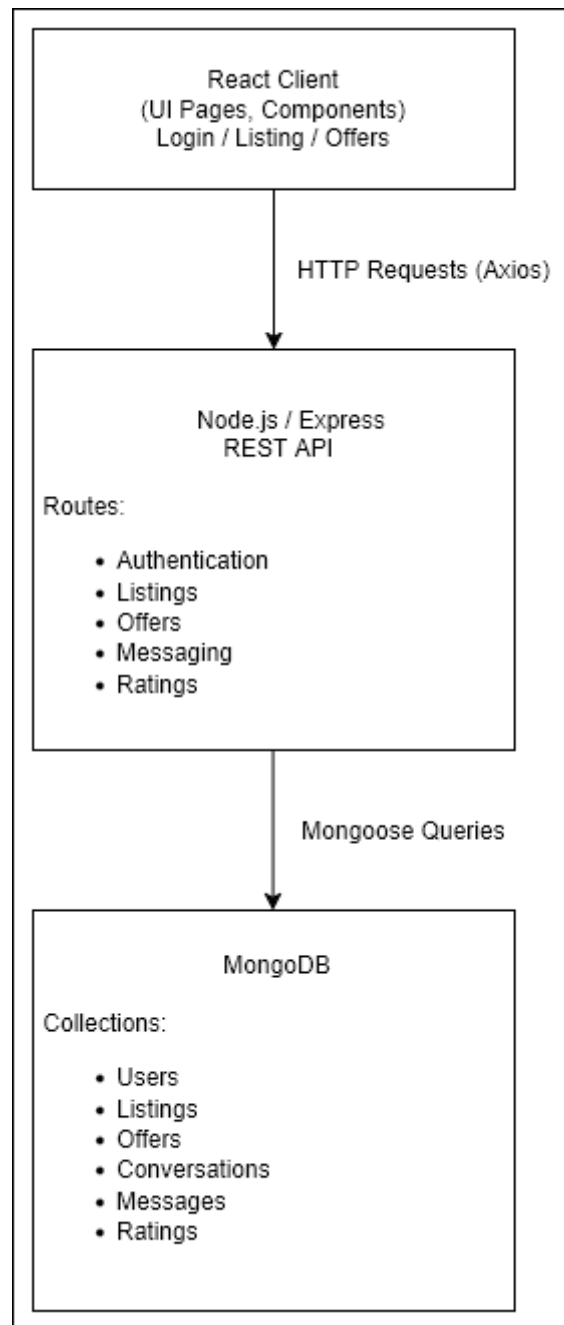
- React frontend application
- Sends HTTP requests to backend API

server/

- Express backend API
- Handles business logic
- Communicates with MongoDB

Environment configuration is handled through a .env file in the server directory.

3. System Architecture



The LionsList system follows a three-tier web architecture consisting of a presentation layer, an application layer, and a data layer.

- The React frontend provides the user interface and communicates with the backend through HTTP requests using Axios.
 - The Express backend processes these requests, applies business logic, and interacts with the MongoDB database through Mongoose models.
 - MongoDB stores persistent application data such as users, listings, offers, conversations, and ratings.
-

4. System Decomposition

4.1 Frontend Components

- Authentication pages (login / register)
- Browse listings page
- Listing detail page
- Create listing form
- My Listings page
- Sent Offers page
- User profile page
- Messaging interface
- Navigation bar

These components communicate with the backend using Axios API requests.

4.2 Backend Components

Backend functionality is implemented using Express routes and controllers.

Major backend modules include:

Authentication

- Register users
- Login users
- Logout functionality

Listings

- Create listings
- Edit listings
- Delete listings
- View listings (with filtering by category, course title, keyword, and status)
- Fetch listings for a specific user

Offers

- Submit offers
- Accept or reject offers
- Cancel offers
- View offers made by a user
- View offers received on a user's listings

Messaging

- Send messages
- Retrieve conversation threads

Ratings

- Store ratings
- Calculate user reputation

4.3 Database Models

The system stores data in MongoDB using Mongoose schemas.

Model	Description
User	Stores account credentials and rating statistics.
Listing	Represents an item listed for sale.
Offer	Represents an offer made by a buyer on a listing.
Conversation	Represents a message thread between buyer and seller.
Message	Individual messages inside a conversation.
Rating	Stores rating scores after completed transactions.

5. CRC Cards

Class Name: User	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: ● Register user accounts ● Authenticate login ● Store username and email ● Track user ratings ● View profile information	Collaborators: ● Listing ● Offer ● Conversation ● Rating

Class Name: Listing	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: ● Store listing information ● Track listing available status ● Allow seller to edit or delete listing ● Display listing details to buyers	Collaborators: ● User ● Offer ● Conversation

Class Name: Offer	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: ● Store offer amount ● Track offer status ● Allow buyers to cancel offers ● Allow sellers to accept or reject offers	Collaborators: ● User ● Listing ● Rating

Class Name: Conversation	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Manage communication between buyer and seller • Link conversation to listing • Store participants	Collaborators: • User • Listing • Message

Class Name: Message	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Store message text • Track sender • Track timestamp • Attach message to conversation	Collaborators: • Conversation • User

Class Name: Rating	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Store rating score • Link rating to completed transaction • Update user reputation	Collaborators: • User • Offer

6. Error Handling Strategy

The LionsList system includes several mechanisms for detecting and responding to errors that may occur during normal operation.

These errors may originate from user input, authentication failures, network communication issues, or database problems. The system is designed so that failures are handled safely and communicated clearly to the user without exposing unnecessary internal system details.

Error handling occurs at both the backend (Express server) and frontend (React client) layers.

6.1 Invalid User Input

Users may enter invalid or incomplete information when registering accounts, creating listings, or submitting offers.

- Username already exists
- Email address not belonging to the York University domain
- Password shorter than the required length
- Missing listing information (title, description, price)
- Invalid offer amount (non-positive values)

The backend implements input validation using Mongoose schema rules and server-side checks. For example, the User schema ensures:

- usernames are unique
- usernames only contain lowercase letters and numbers
- email addresses must end in *@yorku.ca* or *@my.yorku.ca*

When invalid input is detected:

1. The backend returns an HTTP 400 (Bad Request) response.
2. An error message is included in the response body.
3. The frontend displays the message to the user and prevents form submission until the issue is corrected.

This ensures that invalid data is not stored in the database.

6.2 Authentication and Authorization Errors

Certain actions require a user to be authenticated:

- creating a listing
- sending a message

- submitting an offer
- accessing user-specific pages (My Listings, Sent Offers) without authentication

If a user attempts to perform these actions without being logged in, the backend returns an HTTP 401 (Unauthorized) response.

The frontend detects this response and redirects the user to the login page.

Logout functionality removes the authentication token stored in the browser, ensuring the user can no longer access protected actions.

6.3 Database Errors

Database errors may occur if MongoDB becomes unavailable or if database operations fail.

- MongoDB connection failure
- duplicate username or email conflicts
- failed data writes

When a database error occurs:

1. The backend logs the error to the server console for debugging.
2. The API returns an HTTP 500 (Internal Server Error) response.
3. The frontend displays a generic error message to the user.

Sensitive system information is not exposed to the user for security reasons.

6.4 Network and API Communication Errors

Communication between the frontend and backend occurs through HTTP requests using Axios.

Possible failures include:

- server unreachable
- network timeouts
- incorrect API endpoint

When such errors occur:

1. Axios detects the request failure.
2. The frontend displays a message indicating that the request failed.
3. The user may retry the operation.

Examples:

- "Unable to connect to server."
- "Request failed. Please try again."

6.5 Data Integrity Protection

To maintain data integrity, the system enforces constraints at multiple levels.

- unique usernames and email addresses in the User schema
- listing ownership verification before editing or deleting a listing
- validation rules for message content and listing information
- ensure only authorized users can modify or delete listings
- ensure only pending offers can be cancelled

These safeguards ensure that invalid or unauthorized data cannot corrupt the system state.

6.6 Logging and Debugging

Server-side errors are logged in the backend console during development. These logs help developers identify issues such as failed database queries or unexpected exceptions.

In a production environment, these logs are able to be redirected to a logging service or monitoring tool.
